

Mukta Mishra

EE 446

Lab 8

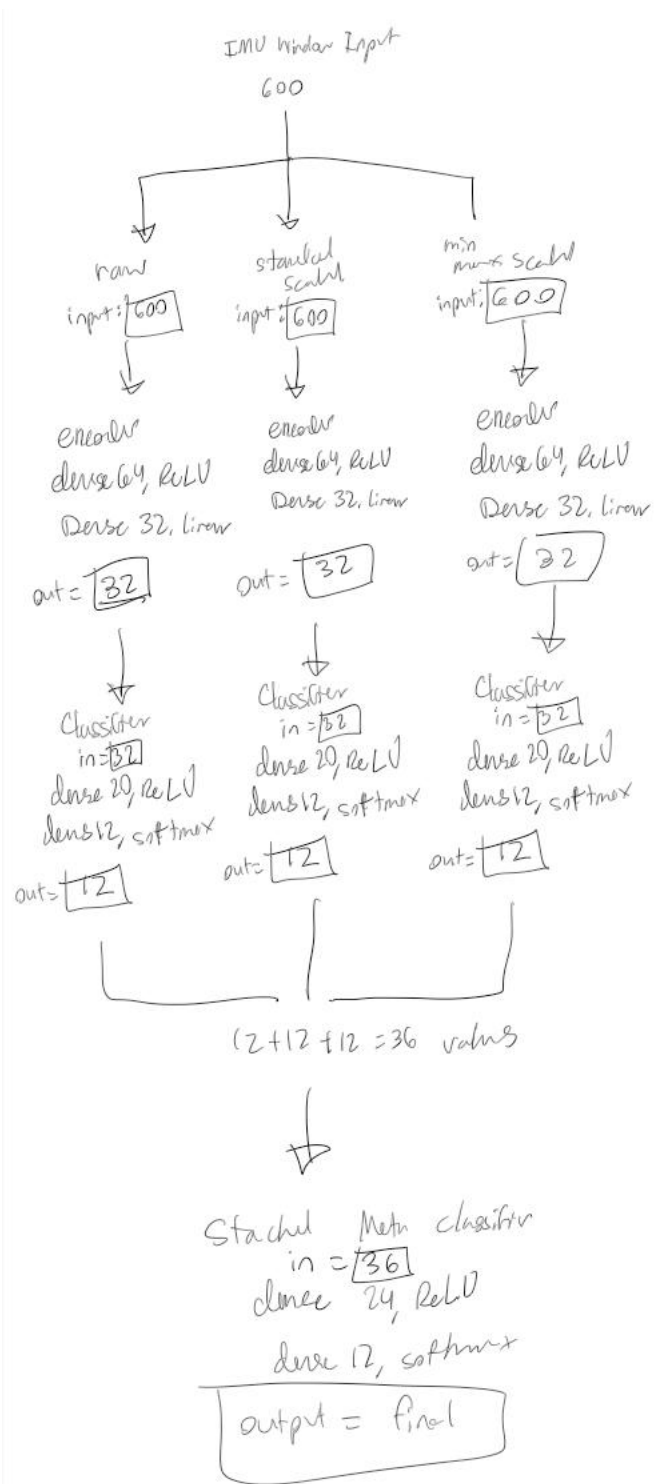
Part 1

<https://studio.edgeimpulse.com/public/1008789/live>

```
No. of classes: 2
Starting inferencing, press 'b' to break
Starting inferencing in 2 seconds...
Sampling...
INFO: HW RFFT failed, FFT size not supported. Must be a power of 2 between 32 and 4096,
(size was 16)Timing: DSP 44 ms, inference 549 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 43 ms, inference 551 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 43 ms, inference 540 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 43 ms, inference 544 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 43 ms, inference 541 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 43 ms, inference 542 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.003906
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 43 ms, inference 549 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
```

Part2

Question 1: Model architecture and ensemble flow



Question 2: Pruning and QAT methodology

The notebook compresses the models using pruning, quantization-aware training, and int8 TensorFlow Lite conversion. First, pruning is applied with a polynomial decay schedule that gradually increases sparsity from 10% to 80%, meaning low-magnitude weights are progressively set to zero during training. After pruning, the pruning wrappers are stripped because they are only needed during pruning training and should not remain in the final model before QAT or deployment. QAT is then applied to the stripped model so the model can learn while simulating the effects of int8 quantization. The notebook also uses mask enforcement during and after QAT because QAT can update weights that were previously pruned, which would reduce the sparsity gained from pruning. The mask-enforcement step keeps those pruned weights at zero. Finally, the model is converted to a fully int8 TFLite model using representative data, which is needed to calibrate the activation ranges and choose good quantization scales. Pruning and int8 quantization help TinyML deployment because they reduce model size, memory use, and computation cost, which is important for a resource-constrained board like the Arduino Nano 33 BLE Sense.

Question 3: Arduino deployment and real-world behavior

```
Confidence score: 0.5508
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.8242, 0.1719, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0039, 0.0000
Standard-scaled model scores: 0.0273, 0.0000, 0.9531, 0.0000, 0.0000, 0.0000, 0.0156, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Min-max-scaled model scores: 0.0000, 0.0000, 0.4375, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0781, 0.0000, 0.0000, 0.4844
Meta-classifier scores: 0.1875, 0.0312, 0.5508, 0.0039, 0.0000, 0.1680, 0.0000, 0.0000, 0.0000, 0.0586, 0.0000, 0.0000
-----
Final prediction: Lying down
Class index: 2
Confidence score: 0.5508
```

When the Arduino Nano 33 BLE Sense was placed flat on the ground, the model consistently predicted “Lying down,” which was generally reasonable for a stationary board condition. The predictions were fairly stable, although the confidence values were not extremely high and the model occasionally assigned some probability to other classes. Some issues observed during deployment included slight prediction instability, sensor noise, and differences between the live Arduino sensor data and the original training dataset. The model also struggled more with distinguishing certain activities such as cycling, running, or climbing stairs, likely because the live motion patterns did not perfectly match the motions used during training. To improve deployment reliability, it would help to collect additional training data directly from a person wearing or carrying the Arduino board so the motion patterns more closely match real human activities. Applying smoothing or averaging across multiple predictions would also help reduce noisy outputs and improve prediction stability.

Question 4: Deployment Behavior on the Arduino

Yes, the deployed model sometimes showed unexpected prediction behavior and would occasionally repeat the same class even when the board was moved differently. This likely happened because the live Arduino sensor data did not perfectly match the original training dataset used in the notebook. The original dataset was collected from human body movements under controlled conditions, while the live deployment used a handheld or stationary Arduino board with different motion patterns and sensor placement. Differences in axis orientation, unit conversion, feature scaling, and board positioning could also affect the input data seen by the model. In addition, some activities such as running, cycling, or climbing stairs are difficult to reproduce accurately by simply moving the board by hand, which can cause the model to confuse similar motions or repeatedly predict the same class. Sensor noise and small inconsistencies in live IMU readings may also contribute to unstable or repetitive predictions during deployment.